LAB 01: Cấu HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG NHÚNG ARMV7

I. THÔNG TIN SINH VIÊN

- **Họ và tên:** Nguyễn Ngọc Quang
- **MSSV:** SE202007
- **Lớp:** FPT University
- **Ngày thực hiện:** 15/09/2026

II. MỤC TIÊU BÀI LAB

- Build Linux Kernel 5.15 cho ARMv7 Cortex-A9
- Build U-Boot 2022.04 bootloader
- Tạo Root Filesystem tối giản với BusyBox
- Boot hệ thống trên QEMU vexpress-a9

III. BOOT SEQUENCE DIAGRAM[200~

IV. KẾT QUẢ ĐẠT ĐƯỢC

PHẦN A: Build Linux Kernel ARMv7

Thành công! ✅

- **Kernel version:** Linux 5.15
- **Architecture:** ARMv7 Cortex-A9
- **Board:** Versatile Express (vexpress-a9)
- **Build command:**

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_defconfig
make menuconfig    # Bật initramfs support
make -j$(nproc) zImage dtbs
```

- **Output files:**
  - `zImage`: 4.8 MB (kernel image)
  - `vexpress-v2p-ca9.dtb`: 14 KB (device tree blob)
- **Compilation flags:** `-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard`
- **Kernel features:** Initramfs support, ARMv7 NEON, VFPv3

PHẦN B: Build U-Boot Bootloader

Thành công! ✅

- **U-Boot version:** 2022.04
- **Board:** vexpress_ca9x4
- **Build command:**

```
make vexpress_ca9x4_defconfig
make -j$(nproc)
```

- **Output files:**
  - `u-boot`: 4.2 MB (executable)
  - `u-boot.bin`: 537 KB (binary)
- **Bootargs:** `console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M`
- **Bootcmd:** `bootz 0x60100000 0x64000000 0x60000000`
- **Features:** QEMU vexpress-a9 support, Serial console

PHẦN C: Build BusyBox & Tạo Initramfs

Thành công! ✅

BusyBox Build:

- **BusyBox version:** 1.35.0
- **Build command:**

```
make defconfig
echo "CONFIG_STATIC=y" >> .config
make -j$(nproc)
make install
```

- **Output:** `_install/` directory (1.6 MB)
- **Linking:** Static linking (không cần shared libraries)
- **Applets:** 300+ utilities

Root Filesystem Structure:~

Initramfs Archive:

- **Create command:**

```
find . | cpio -H newc -o | gzip > initramfs.cpio.gz
```

- **Output:** `initramfs.cpio.gz` (1.1 MB)
- **Format:** CPIO newc + gzip compression
- **Boot parameter:** `rdinit=/sbin/init`

V. CÁC LỆNH CHÍNH TRONG QUÁ TRÌNH BUILD

1. Thiết lập môi trường cross-compile:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export KCFLAGS="-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard"
```

2. Build Kernel:

```
cd ~/embedded_lab1/kernel/linux-5.15
make vexpress_defconfig
make menuconfig
make -j$(nproc) zImage dtbs
cp arch/arm/boot/zImage ~/embedded_lab1/output/
cp arch/arm/boot/dts/vexpress-v2p-ca9.dtb ~/embedded_lab1/output/
```

3. Build U-Boot:

```
cd ~/embedded_lab1/uboot/u-boot-2022.04
make vexpress_ca9x4_defconfig
make -j$(nproc)
cp u-boot ~/embedded_lab1/output/
```

4. Build BusyBox:

```
cd ~/embedded_lab1/rootfs/busybox-1.35.0
make defconfig
sed -i 's/CONFIG_TC=y/CONFIG_TC=n/' .config
echo "CONFIG_STATIC=y" >> .config
make -j$(nproc)
make install
```

5. Tạo Initramfs:

```
cd ~/embedded_lab1/rootfs/initramfs
find . | cpio -H newc -o | gzip > ~/embedded_lab1/output/initramfs.cpio.gz
```

6. Boot trên QEMU:

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -initrd ~/embedded_lab1/output/initramfs.cpio.gz \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```

VI. PHÂN TÍCH & CÂU HỎI

Câu 1: Tại sao phải sử dụng Cross-Compiler?

Trả lời: Vì máy host chạy x86-64 nhưng target board là ARMv7, nên cần cross-compiler (arm-linux-gnueabihf-) để biên dịch code từ x86 thành ARM instruction set. Nếu dùng native compiler, code sẽ không chạy được trên ARM.

Câu 2: Sự khác biệt giữa gnueabi và gnueabihf?

Trả lời: - **gnueabi:** Floating Point Soft (FPU emulation bằng software) - **gnueabihf:** Floating Point Hard (FPU sử dụng phần cứng VFPv3)

Cortex-A9 có FPU phần cứng, nên dùng gnueabihf nhanh hơn 3-4 lần.

Câu 3: Tại sao phải static link BusyBox?

Trả lời: Vì initramfs là standalone filesystem, không có shared libraries libc.so.6. Static linking nhúng tất cả libraries vào binary, giúp chạy độc lập mà không cần libc trong initramfs.

Câu 4: Memory layout của vexpress-a9 là gì?

Trả lời: ### Câu 5: Bootargs có ý nghĩa gì? **Trả lời:** —

VII. KHÓKHĂN GẶP PHẢ VÀ CÁCH GIẢI QUYẾT

Lỗi 1: “No rule to make target ‘zImage’”

Nguyên nhân: Biến môi trường `ARCH` bị mất sau mỗi khi mở terminal mới. **Giải quyết:**

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Lỗi 2: “Error: selected processor does not support ‘dmbisht’ in ARM mode”

Nguyên nhân: BusyBox TC (Traffic Control) chứa instruction ARMv8, không hỗ trợ ARMv7. **Giải quyết:** Disable TC:

```
sed -i 's/CONFIG_TC=y/CONFIG_TC=n/' .config
```

Lỗi 3: BusyBox linking error sau khi config initramfs vào kernel

Nguyên nhân: Kernel config nhập nhầm đường dẫn initramfs cpio.gz. **Giải quyết:** Build kernel bình thường, không nhúng initramfs vào kernel, dùng bootloader pass initramfs.

Lỗi 4: Device node permission error khi copy rootfs

Nguyên nhân: /dev/console, /dev/null cần quyền root để tạo. **Giải quyết:** Dùng `sudo mknod` để tạo device nodes.

VIII. NHẬN XÉT & KẾT LUẬN

Những điều học được:

1. **Cross-compilation:** Quy trình xây dựng embedded Linux từ source
2. **ARM Architecture:** Hiểu rõ ARMv7 Cortex-A9, VFPv3 FPU, instruction set
3. **Bootloader sequence:** Power ON → ROM → U-Boot → Kernel → Init → Shell
4. **Root filesystem:** Tạo minimal filesystem với BusyBox static linked
5. **Initramfs:** Cpio archive format, compression, boot parameters

Kỹ năng thực hành:

- ✅ Cross-compile Linux kernel cho ARM
- ✅ Build U-Boot bootloader
- ✅ Build BusyBox from source
- ✅ Tạo initramfs archive
- ✅ Cấu hình boot sequence trên QEMU

Ứng dụng thực tế:

Quy trình này được sử dụng trong: - Embedded Linux development (router, camera, IoT devices) - SoC porting (porting Linux lên chip mới) - Bootloader development - Kernel porting và optimization

---

IX. FILE OUTPUT & CẤUTRÚC NỘPBÀI

X. TÀI LIỆU THAM KHẢO

- Linux Kernel Documentation: https://kernel.org/doc/
- U-Boot Documentation: https://www.denx.de/wiki/U-Boot/
- BusyBox: https://busybox.net/
- ARM Architecture Reference Manual
- QEMU vexpress-a9 device tree

Báo cáo được hoàn thành vào ngày: 15/09/2026

Trạng thái: ✅ HOÀN THÀNH
